Compte Rendu du Mini-Projet Android Kotlin
Application de Facturation

Préparé par : AMEZIAN Aziz
Module : Développement Mobile
Encadrant : M. BOUABID

1. Cahier des charges :

- Le projet consiste à développer une application mobile Android Kotlin permettant de :

- Ajouter des articles avec leur libellé et leur prix hors taxe (HT) ;
- Sélectionner un taux de TVA (5 %, 10 %, 15 %, 20 %) ;
- Calculer et afficher automatiquement le prix TTC ;
- Afficher les articles ajoutés dans un tableau ;
- Sauvegarder et charger les articles à l'aide d'un fichier objet binaire ;
- Gérer les erreurs de saisie (champs vides, données non numériques).

2. Outils logiciels utilisés :

- Android Studio ;
- Kotlin comme langage principal ;
- Fichiers XML pour l'interface graphique ;
- Android SDK API 25+
- Mon smartphone HUAWEI Y9 pour tests finaux ;

3. Étapes de codage et structure du projet :

➢ Classe Article.kt :

```
package com.example.facturationapp
import java.io.Serializable
data class Article( val libelle: String, val prixHT: Double) : Serializable
```

- Cette classe représente un article avec deux attributs : **libelle** et **prixHT** ;
- Elle implémente Serializable pour permettre l'enregistrement dans un fichier binaire.

➢ Classe Panier.kt :

```
package com.example.facturationapp
import android.content.Context
import java.io.*
class Panier {
   var art: ArrayList<Article> = ArrayList()
   constructor()
   fun save(context: Context) {
      try {
         val fileOutput = context.openFileOutput("data", Context.MODE_PRIVATE)
         val objectOutput = ObjectOutputStream(fileOutput)
         objectOutput.writeObject(art)
         objectOutput.close()
         fileOutput.close()
      } catch (e: Exception) {
         e.printStackTrace()
      }
   }
```

```
fun load(context: Context) {
    try {
        val fileInput = context.openFileInput("data")
        val objectInput = ObjectInputStream(fileInput)
        @Suppress("UNCHECKED_CAST")
        art = objectInput.readObject() as ArrayList<Article>
        objectInput.close()
        fileInput.close()
    } catch (e: Exception) {
        e.printStackTrace()
    }
}
```

Gère une liste d'articles avec deux fonctions :

- **save()** : enregistre la liste dans un fichier data ;
- **load()** : recharge les articles à partir de ce fichier.

➢ Classe MainActivity.kt :

But principal de la classe :
C'est la classe principale de l'application, elle gère :

- L'interface utilisateur
- Les interactions avec l'utilisateur
- Les calculs de prix TTC
- La sauvegarde et le chargement du panier via la classe Panier
- Les vérifications des erreurs de saisie

❖ *Attributs de la classe :*

- **editLibelle** : Champ de saisie du nom de l'article ;
- **editPrixHT** : Champ de saisie du prix hors taxe ;
- **spinnerTVA** : Liste déroulante pour choisir le taux de TVA ;
- **textPrixTTC** : Affiche le prix TTC de l'article courant ;
- **textTotalTTC:** Affiche le total TTC de tous les articles ;
- **tableArticles :** Table affichant tous les articles ajoutés ;
- **Panier** : Objet de type Panier contenant tous les articles.

❖ *Liaison avec l'IHM (XML) :*

La méthode findViewById(...) est utilisée pour **lier les éléments XML aux attributs Kotlin**, par exemple :

```
editLibelle = findViewById(R.id.editLibelle)
```

Cela permet à Kotlin de contrôler et modifier les éléments affichés à l'écran.

❖ *Méthode onCreate() :*

C'est le **point de départ de l'activité**. Elle :

1. Initialise tous les champs de l'IHM ;
2. Charge les articles précédemment sauvegardés (panier.load(this)) ;
3. Configure le Spinner pour afficher les taux de TVA ;
4. Affiche tous les articles déjà présents ;
5. Attribue des actions aux boutons (ajouter, enregistrer, annuler) ;

❖ ***Méthode ajouterArticle() :***

Cette fonction est appelée quand l'utilisateur clique sur le bouton "**Ajouter Article**".
Étapes internes :

1. Récupère les données saisies (libelle, prixHT) ;
2. Vérifie si les champs sont vides → Affiche un message d'erreur si oui ;
3. Vérifie si prixHT est un nombre → Sinon affiche un message ;
4. Calcule le prix TTC :
   val prixTTC = prixHT + (prixHT * tva / 100)
5. Affiche le prix TTC de l'article ;
6. Crée un objet Article, l'ajoute au panier, l'ajoute à la table, met à jour le total ;
7. Réinitialise les champs de saisie ;

Gestion des erreurs dans cette fonction :

- **Champ vide** :

```
if (libelle.isBlank() || prixHTStr.isBlank())
→ Affiche un Toast : "Champs vides !"
```

- **Valeur non numérique** :

```
val prixHT = prixHTStr.toDoubleOrNull()
→ Si null, affiche : "Prix HT invalide !"
```

❖ ***Méthode updateTotalTTC() :***

- Parcourt tous les articles ;
- Recalcule leur prix TTC selon le taux de TVA sélectionné ;
- Affiche la **somme totale TTC** dans textTotalTTC .

❖ ***Méthode addArticleToTable() :***

- Crée une ligne (TableRow) dans le tableau ;
- Ajoute 3 colonnes : Libellé, Prix HT, Prix TTC ;
- Met à jour dynamiquement l'affichage dans TableLayout ;

❖ ***Méthode createTextView(text: String) :***

- Fonction utilitaire qui crée un élément TextView avec du texte + un peu de style (padding) ;
- Utilisé pour simplifier l'ajout d'une cellule dans le tableau.

❖ ***Bouton Enregistrer (btnEnregistrer) :***

- Appelle la méthode panier.save(this) ;
- Sauvegarde tous les articles dans un fichier binaire (data) ;
- Affiche un message Toast : *"Panier enregistré !"*.

❖ ***Bouton Annuler (btnAnnuler) :***

- Vide les champs de saisie (libellé, prix) ;
- Efface le prix TTC affiché.

➢ **L'IHM : activity_main.xml :**

1. **ImageView**

```
<ImageView
   android:src="@drawable/icone" />
```

Affiche une icône en haut à gauche de l'écran.

2. **Titre principal**

```
<TextView
   android:text="Application de Facturation" />
```

Titre en gras et centré. C'est le nom de l'application affiché à l'utilisateur.

3. **CardView : Zone de saisie des articles**

```
<androidx.cardview.widget.CardView>
  <EditText id="editLibelle" />
  <EditText id="editPrixHT" />
  <Spinner id="spinnerTVA" />
  <TextView id="textPrixTTC" />
</androidx.cardview.widget.CardView>
```

- editLibelle : champ de saisie du nom de l'article
- editPrixHT : champ numérique pour le prix HT
- spinnerTVA : menu déroulant pour sélectionner la TVA
- textPrixTTC : affiche le prix TTC calculé

La CardView rend l'ensemble plus lisible et professionnel.

4. **TableLayout : affichage des articles ajoutés**

```
<TableLayout id="tableArticles">
  <TableRow>...</TableRow>
</TableLayout>
```

- Affiche chaque article sous forme de ligne :
  - Libellé
  - Prix HT
  - Prix TTC
- Une ligne d'en-tête est déjà définie statiquement.

Les lignes suivantes sont ajoutées dynamiquement depuis **MainActivity.kt**.

5. **TextView : Total TTC**

```
<TextView id="textTotalTTC" />
```

Affiche le **total TTC** de tous les articles.

6. **Boutons**

```
<Button id="btnAjouter" />
<Button id="btnEnregistrer" />
<Button id="btnAnnuler" />
```

- btnAjouter : ajoute un article à la table
- btnEnregistrer : sauvegarde les articles dans un fichier
- btnAnnuler : vide les champs de saisie

Chaque bouton est stylisé avec une couleur (backgroundTint) correspondant à sa fonction :

- **Vert** pour "Ajouter"
- **Bleu** pour "Enregistrer"
- **Rouge** pour "Annuler"

---

7. **Signature**

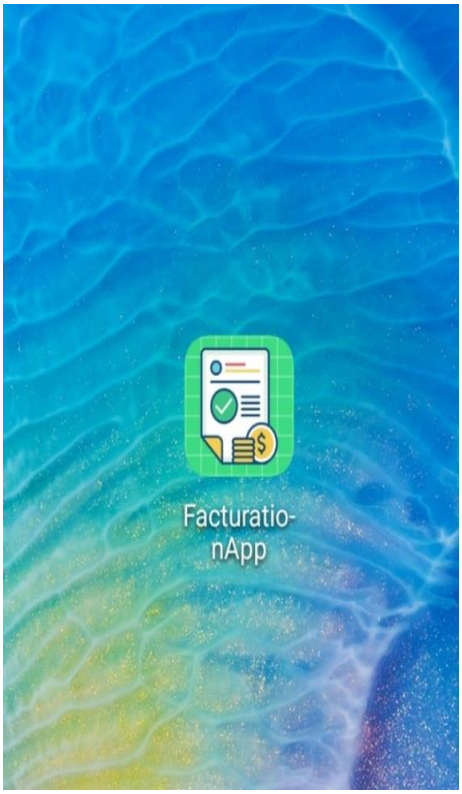
```
<TextView id="signature" />
```

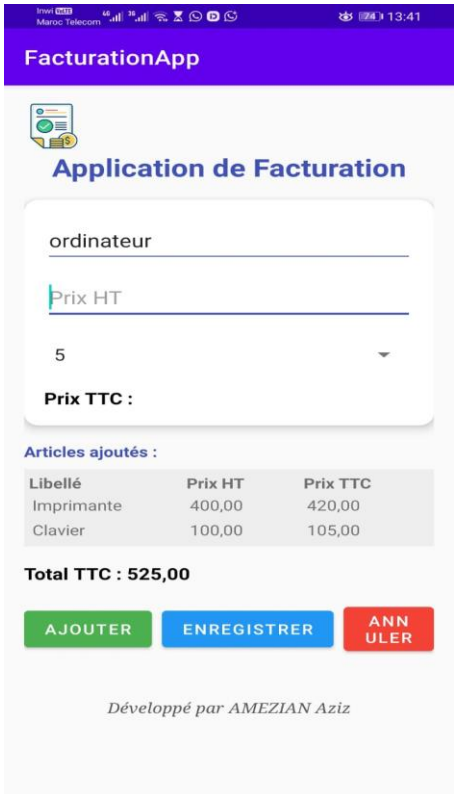
Ajoute un message personnel :

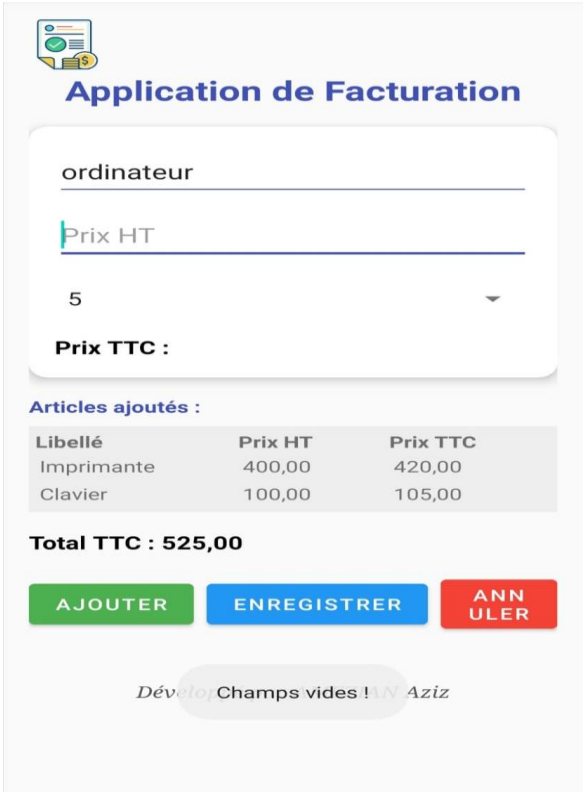
🖉 *"Développé par AMEZIAN Aziz"*

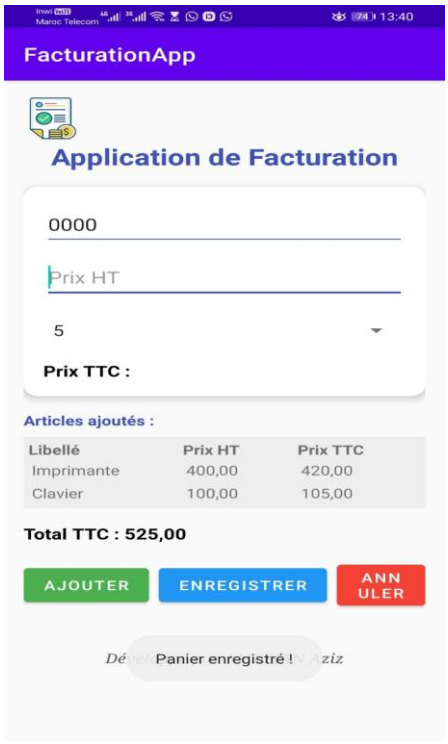
3. Résultats obtenus :

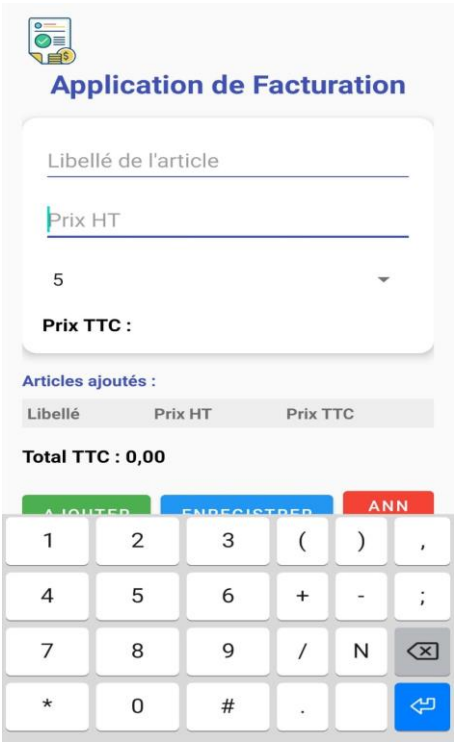
- ***Interfaces obtenues :***











- Page de saisie
- Article ajouté dans la table
- Total TTC affiché
- Données sauvegardées, puis rechargées avec succès

4. Tests effectués :

- Ajout de plusieurs articles avec TVA différentes ✓ ;
- Tests avec champs vides ✗ → Message d'erreur affiché ✓ ;
- Fermeture et relance de l'application → articles rechargés depuis le fichier data ✓ ;
- Test sur smartphone Android physique avec .apk ✓.
- Test d'icone.

5. Conclusion :

Ce mini-projet m'a permis de :

- Comprendre la structure d'un projet Android classique (activité, interface, logique métier) ;
- Manipuler des fichiers binaires avec Serializable ;
- Créer une interface utilisateur intuitive avec XML ;
- Gérer les événements et la validation des données côté Kotlin ;
- Réaliser un cycle complet de développement mobile, du codage au test sur smartphone.

6. Vision de continuité :

- Ajouter la suppression ou modification d'articles ;
- Exporter les données en fichier PDF ou CSV ;
- Améliorer l'interface avec Material Design ;
- Ajouter une base de données SQLite au lieu d'un fichier binaire pour plus de fiabilité…